

NUR Hand-in 2

Berend Nieuwhof

March 19, 2025

Abstract

In this document, the solutions to the second hand-in exercise are presented.

1 Exercise 1: Satellite galaxies

In this section, the solutions to the first exercise are presented.

Our script is given by:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import copy
4
5 def n(x, A, Nsat, a, b, c):
6     return A*Nsat*((x/b)**(a-3))*np.exp(-(x/b)**c)
7
8 def N(x, A, Nsat, a, b, c):
9     return n(x, A, Nsat, a, b, c)*4*np.pi*x**2
10
11 def romberg_interval(func, a, b, order, *args):
12     '''Calculates the integral of a function from a to b, using
13     some order of Richardson extrapolation'''
14     r = np.zeros(order)
15
16     # calculate r0
17     h = (b - a)
18     r[0] = 0.5 * h * (func(a, *args) - func(b, *args))
19     Np = 1
20     for i in np.arange(1, order-1): # calculate approximations
21         r[i] = 0
22         delta = h
23         h = 0.5 * h
24         x = a + h
25         for n in range(Np):
26             r[i] += func(x, *args)
27             x += delta
28         r[i] = 0.5*(r[i-1] + delta*r[i])
29         Np = 2*Np
30     Np = 1
31     for i in np.arange(1, order-1): # combine approximations. r[0] holds the best one
32         Np = 4*Np
33         for j in np.arange(0, order-i):
34             r[j] = (Np * r[j+1] - r[j]) / (Np - 1)
35
36     return r
37
38 def romberg(func, x, order, *args):
39     '''with x a linspace. calculates the integral over a range given by x.
40     len(x) is the number of function evaluations we use to integrate.'''
41     result = 0
42     for i in range(len(x)-1):
43         a = x[i]
44         b = x[i+1]
45         result += romberg_interval(func, a, b, order, *args)[0]
46     return result
```

```

47 |
48 | A=1. # to be computed
49 | Nsat=100
50 | a=2.4
51 | b=0.25
52 | c=1.6
53 |
54 | # We want to integrate the function (with A = 1) to find what we need to set A to
55 | # to find <Nsat> = 100. A will be 100 divided by the value of the integral.
56 |
57 | # Because n(x) is radially symmetric, we can use the fact that
58 | # dV = 4pi*r**2 dr to write down the integral.
59 |
60 | x = np.linspace(0.00001, 5, 15)
61 | integ_result = romberg(N, x, 10, A, 100, a, b, c)
62 | new_A = Nsat / integ_result
63 | print('A found by integral normalisation:', new_A)
64 |
65 | check = romberg(N, x, 10, new_A, Nsat, a, b, c)
66 | check = Nsat / check
67 | print('Sanity check:', check)
68 |
69 | def xor_shift(seed, a1 = 21, a2 = 35, a3 = 4):
70 |     '''Performs 64bit xor shift on the seed number.'''
71 |     state = np.uint64(seed)
72 |     a1 = np.uint64(a1)
73 |     a2 = np.uint64(a2)
74 |     a3 = np.uint64(a3)
75 |
76 |     state = state ^ (state >> a1)
77 |     state = state ^ (state << a2)
78 |     state = state ^ (state >> a3)
79 |     return state
80 |
81 | def mult_w_carry(seed, a = 4294957665):
82 |     '''Performs multiply with carry, base 2**32'''
83 |     state = np.uint64(seed)
84 |     a = np.uint64(a)
85 |
86 |     state = a*(state & np.uint64(2**32 - 1)) + (state >> np.uint64(32))
87 |     number = state & np.uint64(2**32 - 1)
88 |
89 |     return number, state
90 |
91 | def rng_int(seed, size):
92 |     state = np.uint64(seed)
93 |     result = np.zeros(size)
94 |     for i in range(size):
95 |         xor_state = xor_shift(state)
96 |         if xor_state < 2**32:
97 |             number, mult_state = mult_w_carry(xor_state)
98 |         else:
99 |             number, mult_state = mult_w_carry(xor_state & np.uint64(2**32 - 1))
100 |         state = mult_state
101 |         result[i] = number
102 |
103 |     return result
104 |
105 | def rng_float(seed, size, a, b):
106 |     ints = rng_int(seed, size)
107 |     floats = a + (b-a)*(ints / (2**32))
108 |     return floats
109 |
110 | def mergesort(a):
111 |     N = len(a)
112 |     idx = np.arange(0, N, 1)
113 |     def mergesort_helper(idx):
114 |         if len(idx) > 1:
115 |             mid_idx = len(idx)//2
116 |             left = copy.deepcopy(idx[:mid_idx])

```

```

117         right = copy.deepcopy(idx[mid_idx:])
118
119         # recursively sort the left and right arrays
120         mergesort_helper(left)
121         mergesort_helper(right)
122         i, j, k = 0, 0, 0
123         # go through the arrays and check which element of is the smallest between
124         them, then add that one to a
125         while i < len(left) and j < len(right):
126             if a[left[i]] < a[right[j]]:
127                 idx[k] = left[i]
128                 i += 1
129             else:
130                 idx[k] = right[j]
131                 j += 1
132                 k += 1
133         # if left or right still hold elements after one of the lists is ran through
134         , put them into a in order
135         while i < len(left):
136             idx[k] = left[i]
137             i += 1
138             k += 1
139         while j < len(right):
140             idx[k] = right[j]
141             j += 1
142             k += 1
143
144         mergesort_helper(idx)
145         return idx
146
147 #Plot of histogram in log-log space with line (question 1b)
148 xmin, xmax = 10**-4, 5
149 N_generate = 10000
150
151 # Make the analytical function
152 relative_radius = np.linspace(xmin, xmax, 10000)
153 analytical_function = N(relative_radius, new_A, Nsat, a, b, c) / Nsat
154
155 # find the maximum value by sorting
156 sorting_idx2 = mergesort(analytical_function)
157 sorted_analytical = analytical_function[sorting_idx2]
158 max_val2 = sorted_analytical[-1]
159
160 # Do rejection sampling for r.
161 Pu3 = 10**rng_float(44, N_generate*10, np.log10(xmin), np.log10(xmax))
162 Pu4 = rng_float(33, N_generate*10, 0, max_val2)
163 P_x = N(Pu3, new_A, Nsat, a, b, c) / Nsat
164
165 rej_result = []
166 for i in range(N_generate*10):
167     if Pu4[i] < P_x[i]:
168         rej_result.append(Pu3[i])
169         if len(rej_result) == 10000:
170             break
171 rej_result = np.array(rej_result) # take the first 10000 elements
172
173 #21 edges of 20 bins in log-space
174 edges = 10**np.linspace(np.log10(xmin), np.log10(xmax), 21)
175 hist = np.histogram(rej_result, bins=edges)[0]
176 hist_scaled = Nsat * hist / (len(Pu3)) # normalize by the amount of samples originally
177 used.
178
179 hist_px = np.histogram(P_x, bins = edges)[0]
180 hist_px_scaled = Nsat * hist / (len(P_x))
181
182 fig1b, ax = plt.subplots()
183 # ax.stairs(hist_px_scaled, edges = edges, fill=False, label = 'calculated Px')
184 ax.stairs(hist_scaled, edges=edges, fill=False, label='Satellite galaxies')
185 plt.plot(relative_radius, analytical_function, 'r-', label='Analytical solution')

```

```

184 ax.set(xlim=(xmin, xmax), ylim=(10**(-3), 10), yscale='log', xscale='log',
185         xlabel='Relative radius', ylabel='Number of galaxies')
186 ax.legend()
187 plt.savefig('my_solution_1b.png', dpi=600)
188 plt.close()
189
190 # We'll achieve this by generating a random sequence using our random number generator.
191 # We will then sort the array and thus generate a shuffle index which we then apply to
192 # the samples from above.
193 rnd_seq = rng.float(69, 10000, 0, 1)
194 rnd_shuffle_idx = mergesort(rnd_seq)
195
196 # Select 100 random satellite galaxies by applying the index to the samples
197 # and then selecting the first 100
198 chosen = rej_result[rnd_shuffle_idx][:100]
199 sorting_idx = mergesort(chosen)
200 chosen_sorted = chosen[sorting_idx]
201
202 fig1c, ax = plt.subplots()
203 ax.plot(chosen_sorted, np.arange(100))
204 ax.set(xscale='log', xlabel='Relative radius',
205        ylabel='Cumulative number of galaxies',
206        xlim=(xmin, xmax), ylim=(0, 100))
207 plt.savefig('my_solution_1c.png', dpi=600)
208
209 # Now we will calculate dn(x)/dx at x = 1.
210
211 def n_deriv(x, A, Nsat, a, b, c):
212     return A*Nsat/b * ((a-3)*(x/b)**(a-4) - c*(x/b)**(a+c-4))*np.exp(-(x/b)**c)
213
214 def central_diff(x, func, h, *args):
215     deriv = (func(x + h, *args) - func(x - h, *args)) / (2*h)
216     return deriv
217
218 def ridder(x, func, h, d, *args, order = 5, tol = 1e-8):
219     r = np.zeros(order)
220     oneDeeth = 1 / d
221     r[0] = central_diff(x, func, h, *args)
222     for o in range(1, order):
223         h = h * oneDeeth
224         r[o] = central_diff(x, func, h, *args)
225     # now iterate to combine the previous
226
227     for j in range(1, order):
228         for i in range(order-1-j):
229             r[i] = (d**(2*j) * r[i+1] - r[i]) / (d**(2*j) - 1)
230     return r[0]
231
232 #Cumulative plot of the chosen galaxies (1c)
233 x = 1
234 h = 0.2
235
236 deriv_x = ridder(x, n, h, 2, new_A, Nsat, a, b, c, order = 10)
237 deriv_x_a = n_deriv(x, new_A, Nsat, a, b, c)
238 print('derivative of n with respect to x')
239 print("with Ridder's method:", deriv_x)
240 print('Analytically:', deriv_x_a)

```

Satellites.py

1.1 1a. Finding A by numerical integration.

In order to find the normalization constant A in the formula given in the problem sheet, we implemented a Romberg integration algorithm. For the integral, we use $dV = 4\pi r^2 dr$. For the rest of this exercise, $n(x)$ will refer to the number density, and $N(x)$ will be the number density multiplied by $4\pi r^2$. The relevant code is given below:

```

2 def n(x, A, Nsat, a, b, c):
3     return A*Nsat*((x/b)**(a-3))*np.exp(-(x/b)**c)
4
5 def N(x, A, Nsat, a, b, c):
6     return n(x, A, Nsat, a, b, c)*4*np.pi*x**2
7
8 def romberg_interval(func, a, b, order, *args):
9     '''Calculates the integral of a function from a to b, using
10     some order of Richardson extrapolation'''
11     r = np.zeros(order)
12
13     # calculate r0
14     h = (b - a)
15     r[0] = 0.5 * h * (func(a, *args) - func(b, *args))
16     Np = 1
17     for i in np.arange(1, order-1): # calculate approximations
18         r[i] = 0
19         delta = h
20         h = 0.5 * h
21         x = a + h
22         for n in range(Np):
23             r[i] += func(x, *args)
24             x += delta
25             r[i] = 0.5*(r[i-1] + delta*r[i])
26         Np = 2*Np
27     Np = 1
28     for i in np.arange(1, order-1): # combine approximations. r[0] holds the best one
29         Np = 4*Np
30         for j in np.arange(0, order-i):
31             r[j] = (Np * r[j+1] - r[j]) / (Np - 1)
32
33     return r
34
35 def romberg(func, x, order, *args):
36     '''with x a linspace. calculates the integral over a range given by x.
37     len(x) is the number of function evaluations we use to integrate.'''
38     result = 0
39     for i in range(len(x)-1):
40         a = x[i]
41         b = x[i+1]
42         result += romberg_interval(func, a, b, order, *args)[0]
43     return result
44
45 A=1. # to be computed
46 Nsat=100
47 a=2.4
48 b=0.25
49 c=1.6
50
51 # We want to integrate the function (with A = 1) to find what we need to set A to
52 # to find <Nsat> = 100. A will be 100 divided by the value of the integral.
53
54 # Because n(x) is radially symmetric, we can use the fact that
55 # dV = 4pi*r**2 dr to write down the integral.
56
57 x = np.linspace(0.00001, 5, 15)
58 integ_result = romberg(N, x, 10, A, 100, a, b, c)
59 new_A = Nsat / integ_result
60 print('A found by integral normalisation:', new_A)
61
62 check = romberg(N, x, 10, new_A, Nsat, a, b, c)
63 check = Nsat / check
64 print('Sanity check:', check)

```

Satellites.py

As can be seen in the code above, romberg_interval calculates the integral between a and b up to some order, and romberg then sums this over many intervals. In this case, we go up to order 10, where we use 15 function evaluations. The output given by this piece of the code is given below. For the sanity check, we calculate the integral again with our calculated value of A, and we see that it is equal to one.

```
1 A found by integral normalisation: 9.211900153463198
2 Sanity check: 1.0000000000000007
```

satellite_output.txt

1.2 1b. Generating satellite positions

For this subquestion, we made use of rejection sampling in log space to sample 10,000 points following the analytical distribution $N(x)dx/\langle N_{sat} \rangle$. The relevant code is given below.

```
1 def xor_shift(seed, a1 = 21, a2 = 35, a3 = 4):
2     '''Performs 64bit xor shift on the seed number.'''
3     state = np.uint64(seed)
4     a1 = np.uint64(a1)
5     a2 = np.uint64(a2)
6     a3 = np.uint64(a3)
7
8     state = state ^ (state >> a1)
9     state = state ^ (state << a2)
10    state = state ^ (state >> a3)
11    return state
12
13 def mult_w_carry(seed, a = 4294957665):
14     '''Performs multiply with carry, base 2**32'''
15     state = np.uint64(seed)
16     a = np.uint64(a)
17
18     state = a*(state & np.uint64(2**32 - 1)) + (state >> np.uint64(32))
19     number = state & np.uint64(2**32 - 1)
20
21     return number, state
22
23 def rng_int(seed, size):
24     state = np.uint64(seed)
25     result = np.zeros(size)
26     for i in range(size):
27         xor_state = xor_shift(state)
28         if xor_state < 2**32:
29             number, mult_state = mult_w_carry(xor_state)
30         else:
31             number, mult_state = mult_w_carry(xor_state & np.uint64(2**32 - 1))
32         state = mult_state
33         result[i] = number
34
35     return result
36
37 def rng_float(seed, size, a, b):
38     ints = rng_int(seed, size)
39     floats = a + (b-a)*(ints / (2**32))
40     return floats
41
42 def mergesort(a):
43     N = len(a)
44     idx = np.arange(0, N, 1)
45     def mergesort_helper(idx):
46         if len(idx) > 1:
47             mid_idx = len(idx)//2
48             left = copy.deepcopy(idx[:mid_idx])
49             right = copy.deepcopy(idx[mid_idx:])
50
51             # recursively sort the left and right arrays
52             mergesort_helper(left)
53             mergesort_helper(right)
54             i, j, k = 0, 0, 0
55             # go through the arrays and check which element of is the smallest between
56             # them, then add that one to a
57             while i < len(left) and j < len(right):
58                 if a[left[i]] < a[right[j]]:
```

```

58         idx[k] = left[i]
59         i += 1
60     else:
61         idx[k] = right[j]
62         j += 1
63     k += 1
64     # if left or right still hold elements after one of the lists is ran through
    , put them into a in order
65     while i < len(left):
66         idx[k] = left[i]
67         i += 1
68         k += 1
69     while j < len(right):
70         idx[k] = right[j]
71         j += 1
72         k += 1
73
74     mergesort_helper(idx)
75     return idx
76
77
78 #Plot of histogram in log-log space with line (question 1b)
79 xmin, xmax = 10**-4, 5
80 N_generate = 10000
81
82 # Make the analytical function
83 relative_radius = np.linspace(xmin, xmax, 10000)
84 analytical_function = N(relative_radius, new_A, Nsat, a, b, c) / Nsat
85
86 # find the maximum value by sorting
87 sorting_idx2 = mergesort(analytical_function)
88 sorted_analytical = analytical_function[sorting_idx2]
89 max_val2 = sorted_analytical[-1]
90
91 # Do rejection sampling for r.
92 Pu3 = 10**rng.float(44, N_generate*10, np.log10(xmin), np.log10(xmax))
93 Pu4 = rng_float(33, N_generate*10, 0, max_val2)
94 P_x = N(Pu3, new_A, Nsat, a, b, c) / Nsat
95
96 rej_result = []
97 for i in range(N_generate*10):
98     if Pu4[i] < P_x[i]:
99         rej_result.append(Pu3[i])
100     if len(rej_result) == 10000:
101         break
102 rej_result = np.array(rej_result) # take the first 10000 elements
103
104 #21 edges of 20 bins in log-space
105 edges = 10*np.linspace(np.log10(xmin), np.log10(xmax), 21)
106 hist = np.histogram(rej_result, bins=edges)[0]
107 hist_scaled = Nsat * hist / (len(Pu3)) # normalize by the amount of samples originally
    used.
108
109 hist_px = np.histogram(P_x, bins = edges)[0]
110 hist_px_scaled = Nsat * hist / (len(P_x))
111
112 fig1b, ax = plt.subplots()
113 # ax.stairs(hist_px_scaled, edges=edges, fill=False, label='calculated Px')
114 ax.stairs(hist_scaled, edges=edges, fill=False, label='Satellite galaxies')
115 plt.plot(relative_radius, analytical_function, 'r-', label='Analytical solution')
116 ax.set(xlim=(xmin, xmax), ylim=(10**(-3), 10), yscale='log', xscale='log',
117        xlabel='Relative radius', ylabel='Number of galaxies')
118 ax.legend()
119 plt.savefig('my_solution.1b.png', dpi=600)
120 plt.close()

```

Satellites.py

The analytical function is equal to $N(x) = n(x)4\pi r^2$. We first find the maximum this function takes on in our interval, by evaluating the function at many points (which is already necessary for plotting),

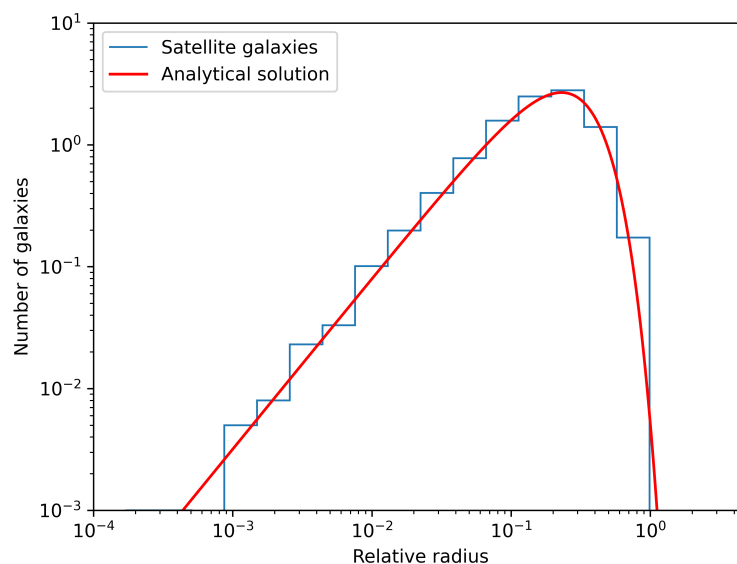
sorting the array of evaluations, and finding the first element of it. We use this maximum as the maximum for our uniform random numbers.

Speaking of uniform random numbers, we generate ours as follows: we feed a number into a xor_shift algorithm, and the result of that is put into a Multiply With Carry random number generator. The MWC returns both a number and a state: the number is output as a random number, and the state is then fed back into the xor_shift algorithm, where the next random number will be generated. This of course generates integers between 0 and 2^{32} , so we use the rng_float function to generate uniform random floats between two values.

We use rejection sampling to sample the distribution, and we do so by sampling our x value in log space, to match the logarithmic bins given by the exercise instructions. We then compare another random number $y = U(0, max)$, with max being the maximum calculated earlier, to the function evaluated at our x value. If y is smaller than x, we accept y as part of our sample. We generate samples until we've reached 10,000, at which point we stop generating them.

The result of this sampling is plotted below. Note that we normalize our histogram by dividing by the total amount of samples generated to reach 10,000 accepted samples, and multiplying by the average number of satellites.

Figure 1: In red, the analytical distribution that we are supposed to sample from is plotted. The blue bars correspond to the logarithmic bins given, and their heights are the number of samples per bin. The bars follow the distribution nicely.



1.3 1c. Random galaxy selection

In this subquestion, we randomly select satellite galaxies from the sample in b in a way that selects every galaxy with equal probability, does not draw the same galaxy twice, and does not reject any draw. The relevant code is included here. All sorting in this exercise is done with mergesort, which can be found in the first code snippet in this exercise.

```

1 # We'll achieve this by generating a random sequence using our random number generator.
2 # We will then sort the array and thus generate a shuffle index which we then apply to
3 # the samples from above.
4 rnd_seq = rng_float(69, 10000, 0, 1)
5 rnd_shuffle_idx = mergesort(rnd_seq)
6
7
8 # Select 100 random satellite galaxies by applying the index to the samples
9 # and then selecting the first 100

```

```

10 chosen = rej_result[rnd_shuffle_idx][:100]
11 sorting_idx = mergesort(chosen)
12 chosen_sorted = chosen[sorting_idx]
13
14 fig1c, ax = plt.subplots()
15 ax.plot(chosen_sorted, np.arange(100))
16 ax.set(xscale='log', xlabel='Relative radius',
17        ylabel='Cumulative number of galaxies',
18        xlim=(xmin, xmax), ylim=(0, 100))
19 plt.savefig('my_solution_1c.png', dpi=600)

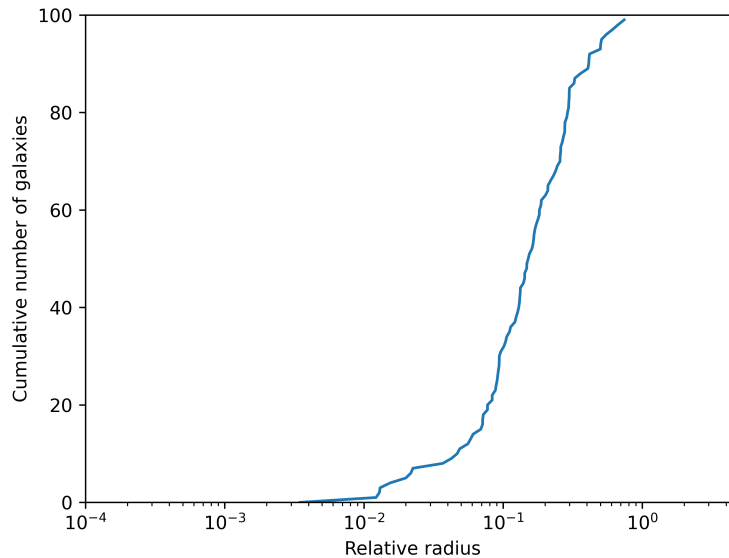
```

Satellites.py

We achieve this by generating an array of 10,000 random numbers, and sorting them to create a random sorting index. We then apply this sorting index to the random sample from b , and select the first 100 elements. This achieves the requirements specified above.

Then, we sort the 100 galaxies from smallest to largest radius using our implementation of mergesort, and plot the number of galaxies within a radius.

Figure 2: We see that the number of galaxies within a radius is strictly increasing, which means the array is sorted correctly.



1.4 1d. Derivative of $n(x)$

In this section, we calculate the derivative $dn(x)/dx$ at $x = 1$ numerically. We used Ridder's algorithm to find the derivative, as seen in the code below.

```

1 # Now we will calculate dn(x)/dx at x = 1.
2
3 def n_deriv(x, A, Nsat, a, b, c):
4     return A*Nsat/b * ((a-3)*(x/b)**(a-4) - c*(x/b)**(a+c-4))*np.exp(-(x/b)**c)
5
6 def central_diff(x, func, h, *args):
7     deriv = (func(x + h, *args) - func(x - h, *args)) / (2*h)
8     return deriv
9
10 def ridder(x, func, h, d, *args, order = 5, tol = 1e-8):
11     r = np.zeros(order)
12     oneDeeth = 1 / d
13     r[0] = central_diff(x, func, h, *args)

```

```

14     for o in range(1, order):
15         h = h * oneDeeth
16         r[o] = central_diff(x, func, h, *args)
17     # now iterate to combine the previous
18
19     for j in range(1, order):
20         for i in range(order-1-j):
21             r[i] = (d**(2*j) * r[i+1] - r[i]) / (d**(2*j) - 1)
22     return r[0]
23
24 #Cumulative plot of the chosen galaxies (1c)
25 x = 1
26 h = 0.2
27
28 deriv_x = ridder(x, n, h, 2, new_A, Nsat, a, b, c, order = 10)
29 deriv_x_a = n_deriv(x, new_A, Nsat, a, b, c)
30 print('derivative of n with respect to x')
31 print("with Ridder's method:", deriv_x)
32 print('Analytically:', deriv_x_a)

```

Satellites.py

The output is given here:

```

1 derivative of n with respect to x
2 with Ridder's method: -0.6264877290898574
3 Analytically: -0.6264877290898926

```

satellite_output.txt

2 Exercise 2: HII-regions

In this section we look at exercise 2, concerning equilibrium temperatures in HII-regions under various heating and cooling influences. We make use of the functions provided, equilibrium1 and equilibrium2. The code for the full exercise is given below:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import copy
4 import timeit
5
6 def bisection(func, brac, *args, abs_tol = 1e-8, frac_tol = 1e-8, max_iter = 50):
7     '''implements the bisection method of root finding.'''
8     i = 0
9     a = brac[0]
10    b = brac[1]
11    # check if a, b is a bracket
12    # assert func(a, *args)*func(b, *args) < 0
13
14    while i < max_iter:
15        # Divide the bracket in 2
16        c = (a+b)/2
17        if func(a, *args)*func(c, *args) < 0:
18            b = c
19        else:
20            a = c
21        i += 1
22        if abs(a-b) < abs_tol or abs((a-b)/a) < frac_tol:
23            return a, i
24    return a, i
25
26 def secant(func, guesses, *args, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 50):
27     '''implements the secant method for root finding.'''
28     i = 2
29     # r will hold all guesses
30     r = np.zeros(max_iter)
31     r[0] = guesses[0]
32     r[1] = guesses[1]

```

```

33     while i < max_iter:
34         r[i] = r[i-1] - ((r[i-1] - r[i-2])/(func(r[i-1], *args) - func(r[i-2], *args)))
35     * func(r[i-1], *args)
36         if abs(r[i] - r[i-1]) < abs_tol or abs((r[i] - r[i-1])/r[i]) < frac_tol:
37             return r[i], i
38             i += 1
39
40     print('maximum secant iterations reached')
41     return r[i-1], i
42
43 def false_position(func, brac, *args, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 50):
44     '''implements the false position method for root finding.'''
45     i = 2
46     r = np.zeros(max_iter)
47     r[0] = brac[0]
48     r[1] = brac[1]
49     # check if a, b is a bracket
50     assert func(r[0], *args)*func(r[1], *args) < 0
51
52     while i < max_iter:
53         r[i] = r[i-1] - ((r[i-1] - r[i-2])/(func(r[i-1], *args) - func(r[i-2], *args)))
54     * func(r[i-1], *args)
55         if func(r[i], *args)*func(r[i-2], *args) < 0:
56             r[i-1] = r[i-2]
57             if abs(r[i] - r[i-1]) < abs_tol or abs((r[i] - r[i-1])/r[i]) < frac_tol:
58                 return r[i], i
59             i += 1
60
61     return r[i-1], i
62
63 ## 2a.
64
65 k=1.38e-16 # erg/K
66 aB = 2e-13 # cm^3 / s
67 Z = 0.015 # unitless
68 psi = 0.929 # unitless
69 Tc = 1e4 # K
70
71 # here no need for nH nor ne as they cancel out
72 def equilibrium1(T,Z,Tc,psi):
73     return psi*Tc*k - (0.684 - 0.0416 * np.log(T/(1e4 * Z*Z)))*T*k
74
75 # test and visualize first
76 temps = np.linspace(1, 1e7, 1000)
77 eqs = equilibrium1(temps, Z, Tc, psi)
78 # plt.plot(temps, eqs)
79 # plt.show()
80
81 brac1 = [1, 1e7] # K
82 midpoint1 = 0.5e7 # mid point of the bracket interval
83 print('-----2a-----')
84 def time_bisection():
85     root1, num_it1 = bisection(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol
86     = 1e-8, max_iter = 50)
87     return root1, num_it1
88
89 def time_secant():
90     root2, num_it2 = secant(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol =
91     1e-8, max_iter = 50)
92     return root2, num_it2
93
94 def time_false_position():
95     root3, num_it3 = false_position(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01,
96     frac_tol = 1e-8, max_iter = 50)
97     return root3, num_it3
98
99 root1, num_it1 = bisection(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1
100 e-8, max_iter = 50)
101 bisection_time = timeit.timeit(lambda : time_bisection(), number = 1)
102 print('\nroot and number of iterations')

```

```

97 print('using bisection:', root1, num_it1)
98 print('function evaluated at root:', equilibrium1(root1, Z, Tc, psi))
99 print('time taken:', bisection_time)
100
101 root2, num_it2 = secant(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e
    -8, max_iter = 50)
102 secant_time = timeit.timeit(lambda : time_secant(), number = 1)
103 print('\nusing secant method:', root2, num_it2)
104 print('function evaluated at root:', equilibrium1(root2, Z, Tc, psi))
105 print('time taken:', secant_time)
106
107 root3, num_it3 = false_position(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01,
    frac_tol = 1e-8, max_iter = 50)
108 false_position_time = timeit.timeit(lambda : time_false_position(), number = 1)
109 print('\nusing false position method:', root3, num_it3)
110 print('function evaluated at root:', equilibrium1(root3, Z, Tc, psi))
111 print('time taken:', false_position_time)
112
113 def equilibrium2(T,Z,Tc,psi, nH, A, xi):
114     return (psi*Tc - (0.684 - 0.0416 * np.log(T/(1e4 * Z*Z)))*T - .54 * ( T/1e4 )**.37 *
        T)*k*nH*aB + A*xi + 8.9e-26 * (T/1e4)
115
116
117 print('-----2b-----')
118 # 2b
119 A = 5e-10 # erg
120 xi = 1e-15 # s**-1
121 brac2 = [1, 1e15] # K
122 k=1.38e-16 # erg/K
123 aB = 2e-13 # cm^3 / s
124 Z = 0.015 # unitless
125 psi = 0.929 # unitless
126 Tc = 1e4 # K
127
128 # test and visualize first
129 temps = np.linspace(1, 1e15, 1000)
130 ne = 1e5
131 eqs = equilibrium2(temps, Z, Tc, psi, ne, A, xi)
132 # plt.plot(temps, eqs)
133 # plt.show()
134
135 for ne in [1e-4, 1, 1e4]:
136     print('\nfor ne = ', ne)
137     root1, num_it1 = bisection(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e
        -10, frac_tol = 1e-10, max_iter = 100)
138     print('root and number of iterations')
139     print('using bisection:', root1, num_it1)
140     print('function evaluated at root:', equilibrium2(root1, Z, Tc, psi, ne, A, xi))
141
142     root2, num_it2 = secant(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10,
        frac_tol = 1e-10, max_iter = 100)
143     print('using secant method:', root2, num_it2)
144     print('function evaluated at root:', equilibrium2(root2, Z, Tc, psi, ne, A, xi))
145
146     root3, num_it3 = false_position(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol
        = 1e-10, frac_tol = 1e-10, max_iter = 100000)
147     print('using false position method:', root3, num_it3)
148     print('function evaluated at root:', equilibrium2(root3, Z, Tc, psi, ne, A, xi))
149
150 print('-----')
151 def smallest_ne():
152     ne = 1e-4
153     root1, num_it1 = bisection(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e
        -10, frac_tol = 1e-10, max_iter = 100)
154     return root1, num_it1
155
156 def middle_ne():
157     ne = 1
158     root2, num_it2 = secant(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10,
        frac_tol = 1e-10, max_iter = 100)

```

```

159     return root2, num_it2
160 def big_ne():
161     ne = 1e4
162     root2, num_it2 = secant(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10,
163                           frac_tol = 1e-10, max_iter = 100)
164     return root2, num_it2
165
166 small_ne_time = timeit.timeit(lambda : smallest_ne(), number = 1)
167 mid_ne_time = timeit.timeit(lambda : middle_ne(), number = 1)
168 big_ne_time = timeit.timeit(lambda : big_ne(), number = 1)
169
170 print('Bisection for smallest n took:', small_ne_time)
171 print('Secant for middle ne took:', mid_ne_time)
172 print('Secant for biggest ne took:', big_ne_time)

```

HII-regions.py

2.1 2a: A simple HII-region

For this exercise, we have to find the root of the equilibrium1 function that was provided. We time and test 3 root finding algorithms, to find the fastest one. The relevant code is given below:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import copy
4 import timeit
5
6 def bisection(func, brac, *args, abs_tol = 1e-8, frac_tol = 1e-8, max_iter = 50):
7     '''implements the bisection method of root finding.'''
8     i = 0
9     a = brac[0]
10    b = brac[1]
11    # check if a, b is a bracket
12    # assert func(a, *args)*func(b, *args) < 0
13
14    while i < max_iter:
15        # Divide the bracket in 2
16        c = (a+b)/2
17        if func(a, *args)*func(c, *args) < 0:
18            b = c
19        else:
20            a = c
21        i += 1
22        if abs(a-b) < abs_tol or abs((a-b)/a) < frac_tol:
23            return a, i
24    return a, i
25
26 def secant(func, guesses, *args, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 50):
27     '''implements the secant method for root finding.'''
28     i = 2
29     # r will hold all guesses
30     r = np.zeros(max_iter)
31     r[0] = guesses[0]
32     r[1] = guesses[1]
33     while i < max_iter:
34         r[i] = r[i-1] - ((r[i-1] - r[i-2])/(func(r[i-1], *args) - func(r[i-2], *args)))
35         * func(r[i-1], *args)
36         if abs(r[i] - r[i-1]) < abs_tol or abs((r[i] - r[i-1])/r[i]) < frac_tol:
37             return r[i], i
38         i += 1
39
40     print('maximum secant iterations reached')
41     return r[i-1], i
42
43 def false_position(func, brac, *args, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 50):
44     '''implements the false position method for root finding.'''
45     i = 2
46     r = np.zeros(max_iter)
47     r[0] = brac[0]

```

```

47     r[1] = brac[1]
48     # check if a, b is a bracket
49     assert func(r[0], *args)*func(r[1], *args) < 0
50
51     while i < max_iter:
52         r[i] = r[i-1] - ((r[i-1] - r[i-2])/(func(r[i-1], *args) - func(r[i-2], *args)))
53         * func(r[i-1], *args)
54         if func(r[i], *args)*func(r[i-2], *args) < 0:
55             r[i-1] = r[i-2]
56             if abs(r[i] - r[i-1]) < abs_tol or abs((r[i] - r[i-1])/r[i]) < frac_tol:
57                 return r[i], i
58             i += 1
59
60     return r[i-1], i
61
62 ## 2a.
63 k=1.38e-16 # erg/K
64 aB = 2e-13 # cm^3 / s
65 Z = 0.015 # unitless
66 psi = 0.929 # unitless
67 Tc = 1e4 # K
68
69 # here no need for nH nor ne as they cancel out
70 def equilibrium1(T,Z,Tc,psi):
71     return psi*Tc*k - (0.684 - 0.0416 * np.log(T/(1e4 * Z*Z)))*T*k
72
73 # test and visualize first
74 temps = np.linspace(1, 1e7, 1000)
75 eqs = equilibrium1(temps, Z, Tc, psi)
76 # plt.plot(temps, eqs)
77 # plt.show()
78
79 brac1 = [1, 1e7] # K
80 midpoint1 = 0.5e7 # mid point of the bracket interval
81 print('-----2a-----')
82 def time_bisection():
83     root1, num_it1 = bisection(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e-8, max_iter = 50)
84     return root1, num_it1
85
86 def time_secant():
87     root2, num_it2 = secant(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e-8, max_iter = 50)
88     return root2, num_it2
89
90 def time_false_position():
91     root3, num_it3 = false_position(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e-8, max_iter = 50)
92     return root3, num_it3
93
94 root1, num_it1 = bisection(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e-8, max_iter = 50)
95 bisection_time = timeit.timeit(lambda : time_bisection(), number = 1)
96 print('\nroot and number of iterations')
97 print('using bisection:', root1, num_it1)
98 print('function evaluated at root:', equilibrium1(root1, Z, Tc, psi))
99 print('time taken:', bisection_time)
100
101 root2, num_it2 = secant(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e-8, max_iter = 50)
102 secant_time = timeit.timeit(lambda : time_secant(), number = 1)
103 print('\nusing secant method:', root2, num_it2)
104 print('function evaluated at root:', equilibrium1(root2, Z, Tc, psi))
105 print('time taken:', secant_time)
106
107 root3, num_it3 = false_position(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e-8, max_iter = 50)
108 false_position_time = timeit.timeit(lambda : time_false_position(), number = 1)
109 print('\nusing false position method:', root3, num_it3)

```

```

110 print('function evaluated at root:', equilibrium1(root3, Z, Tc, psi))
111 print('time_taken', false_position_time)

```

HII-regions.py

We use a bracket of $[1, 10^7]$ K for all three of them. The output is given below:

```

1 -----2a-----
2
3 root and number of iterations
4 using bisection: 32539.12576294411 30
5 function evaluated at root: 3.2802042676833513e-20
6 time taken: 0.00011419799557188526
7 maximum secant iterations reached
8 maximum secant iterations reached
9
10 using secant method: nan 50
11 function evaluated at root: nan
12 time taken: 0.00033218099997611716
13
14 using false position method:, 32539.126737497554 23
15 function evaluated at root: 0.0
16 time_taken 0.00022847299987915903

```

HII-regions_output.txt

As we can see, both bisection and false position find the root, while secant diverges. We also see the number of iterations, next to the value of the root. We see that the time taken by bisection is the smallest, and we should use bisection in this case.

2.2 2b: A more complex HII-region

We now have to find the root of the equilibrium2 function, which is a little more complicated. We evaluate all three methods for each value of n_e , and at the end we print how fast the best method converges for the different values.

```

1 -----2b-----
2
3 for ne = 0.0001
4 root and number of iterations
5 using bisection: 160647887532832.94 36
6 function evaluated at root: 1.3118954069051732e-26
7 maximum secant iterations reached
8 using secant method: nan 100
9 function evaluated at root: nan
10 using false position method:, 160647887536816.06 124
11 function evaluated at root: 0.0
12
13 for ne = 1
14 root and number of iterations
15 using bisection: 33861.300234646915 69
16 function evaluated at root: 1.5869127818352802e-35
17 using secant method: 33861.300235178554 11
18 function evaluated at root: -9.183549615799121e-41
19 using false position method:, 33861.298551236556 100000
20 function evaluated at root: 5.026513802461895e-32
21
22 for ne = 10000.0
23 root and number of iterations
24 using bisection: 10525.886018968453 70
25 function evaluated at root: 1.9976650564114478e-31
26 using secant method: 10525.88601966122 10
27 function evaluated at root: 2.3553508877120796e-37
28 using false position method:, 10525.885169143065 100000
29 function evaluated at root: 2.4525554870977876e-28
30
31 Bisection for smallest n took: 0.0001609920000191778
32 Secant for middle ne took: 9.385299927089363e-05
33 Secant for biggest ne took: 8.058000094024464e-05

```

For small n_e , secant diverges. Bisection and false position both find the root, but bisection is fastest. We choose bisection.

For middle n_e , secant does not diverge. It actually finds the root in only 11 iterations. Bisection also works, but with more iterations, and false position takes a very long time to converge. It takes about 100,000 iterations to reach a good minimum, which makes it the slowest.

For largest n_e , secant again finds the root the fastest. False position is still the slowest, and bisection is pretty good.

We conclude that we should choose bisection for the first value n_e , and secant for the other two values.